

UNIVERSITÀ
DEGLI STUDI
DELL'AQUILA

Lesson 9: Neural Networks

Technologies for Artificial Intelligence

Fabio Antonini, Università degli Studi dell'Aquila

11 December 2026

Agenda

- One neuron is one line, and where one line runs out
- A hidden layer: several lines and a vote
- How a network learns: the training loop, and one step worked by hand
- A network in Keras on handwritten digits, and softmax
- Training in practice: memorising, two remedies, deep sigmoid networks, shuffled pixels

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Exercise 8, before we start

- Nothing is collected: we discuss it, ten minutes
- What counts is the **prediction made before plotting**, and its reason
- The common gap: trusting a two-dimensional plot to show every anomaly
- Today: a network’s high training accuracy, alone, says little

What a network adds

- Every boundary so far had its **shape fixed in advance**
- Logistic regression: a line; a tree: boxes; a support vector machine: its kernel’s curve
- Training only chose **where** that shape went
- A neural network learns **the shape itself**
- It is built from one simple part, the unit, repeated in layers: the layers between input and output are **hidden layers**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Three problems, and a ceiling of 0.97

- **Acceptance test**: 3,000 sensors, pass or fail
- **Two-channel drift**: 800 sensors, correctable or not
- **Digits**: 1,797 handwritten images, 8 × 8 pixels
- The test rig is wrong on **3%** of sensors, so no score can exceed **0.97**
- A hidden layer is worth **39 points** on the sensors, **2.8** on the digits

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The perceptron, and a unit you already have

- The **perceptron** (Rosenblatt, 1958): output 1 if the weighted sum is positive
- Swap that hard step for lesson 4's sigmoid
- The result is one logistic regression unit, `LogisticRegression`
- New today: the unit is a **component**, repeated
- Also called a **unit**, or a **neuron**

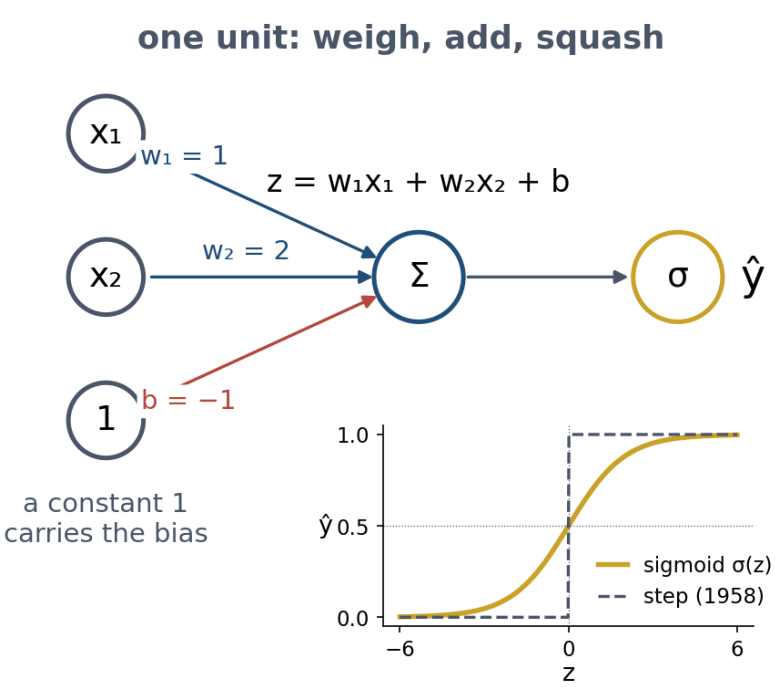
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

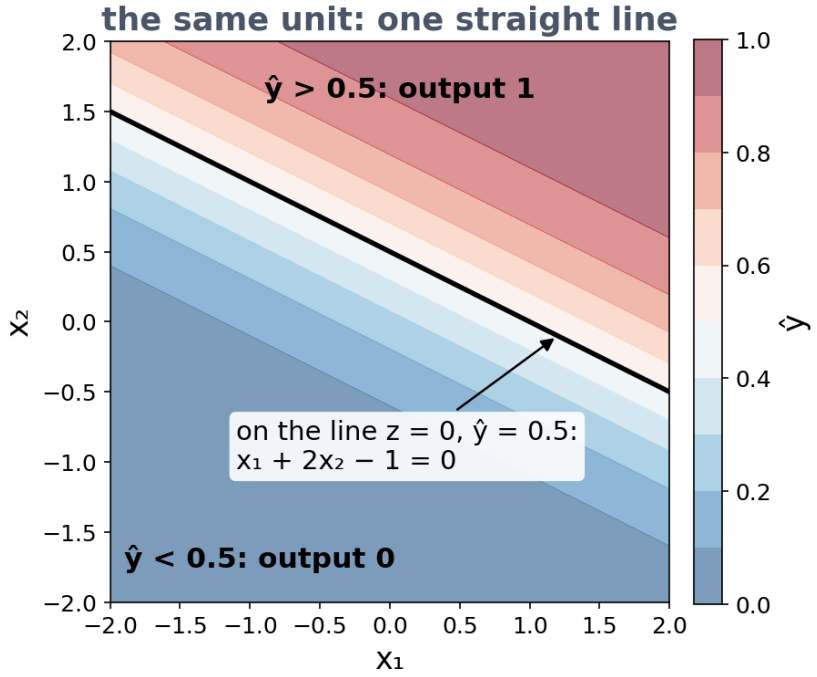
A neuron is a line

$$\hat{y} = \sigma(w^\top x + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

One neuron, drawn





UNIVERSITÀ DEGLI STUDI DELL'AQUILA

One unit, by hand

- Weights $w = (1, 2)$, bias $b = -1$: the unit's line is $x_1 + 2x_2 - 1 = 0$
- Point (1, 1): $z = 1 + 2 - 1 = 2$, and $\hat{y} = \sigma(2) =$ **0.8808**, class 1
- Point (0, 0): $z = -1$, and $\hat{y} =$ **0.2689**, class 0
- Point (1, 0): $z = 0$, and $\hat{y} =$ **0.5000**: on the line, **undecided**
- The unit's verdict is which side of its line a point lies on

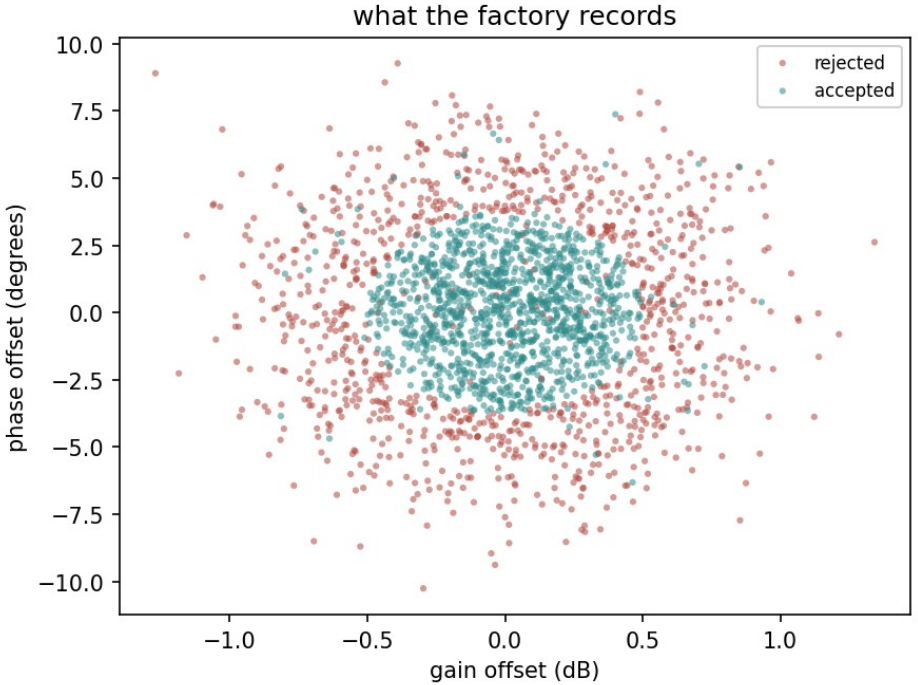
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

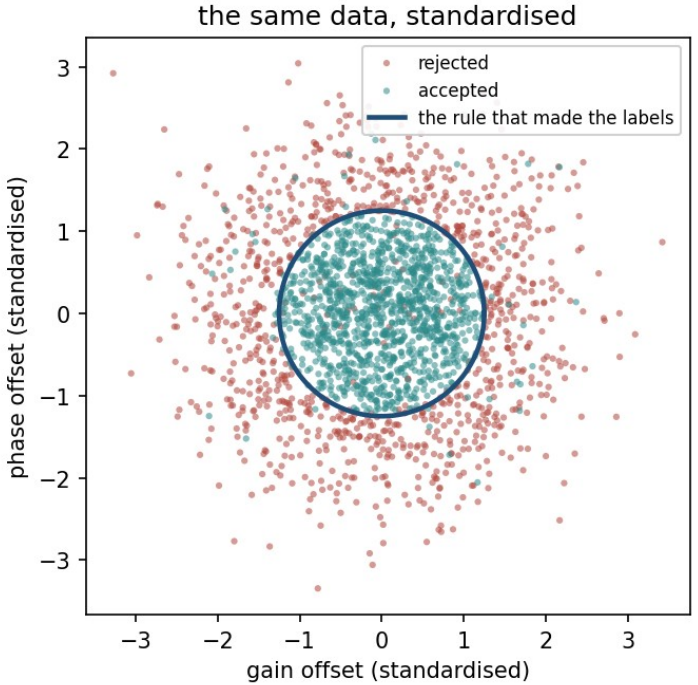
Meridian’s acceptance test

- Each sensor has a gain offset (decibels) and a phase offset (degrees)
- It passes when both offsets are **jointly** small
- 3,000 sensors: `train_test_split` keeps 2,250 to train and 750 to test
- `StandardScaler`, fitted on the 2,250 only, divides each axis by its spread (standard deviation)
- Both tolerances are 1.25 spreads, so a sensor passes inside a **circle of radius 1.25**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

2,250 sensors, raw and standardised





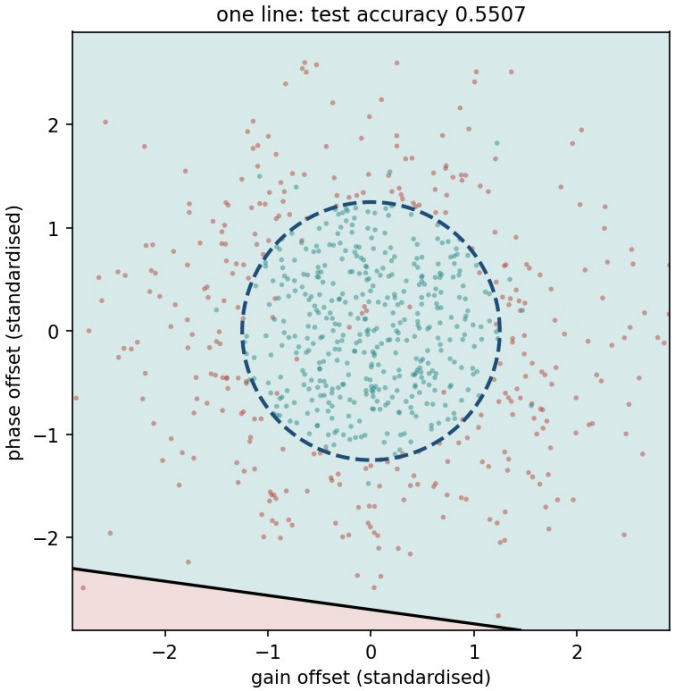
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

What one line is worth

model, scored on the 750 test sensors	accuracy
always predict “accepted”	0.5480
logistic regression: one unit, one line	**0.5507**
the rig’s ceiling	0.9700

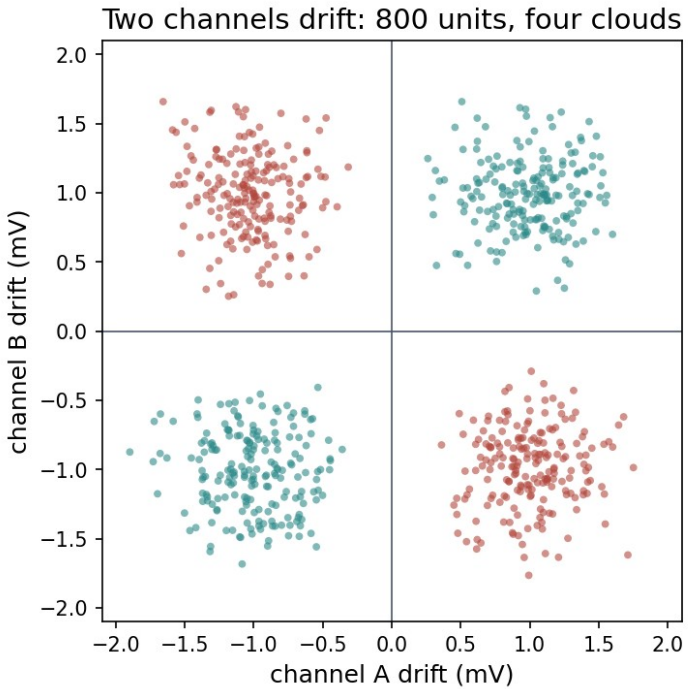
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

No line encloses a region



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

New data: two drifting channels



same sign (common mode): correctable
opposite signs (differential): not correctable

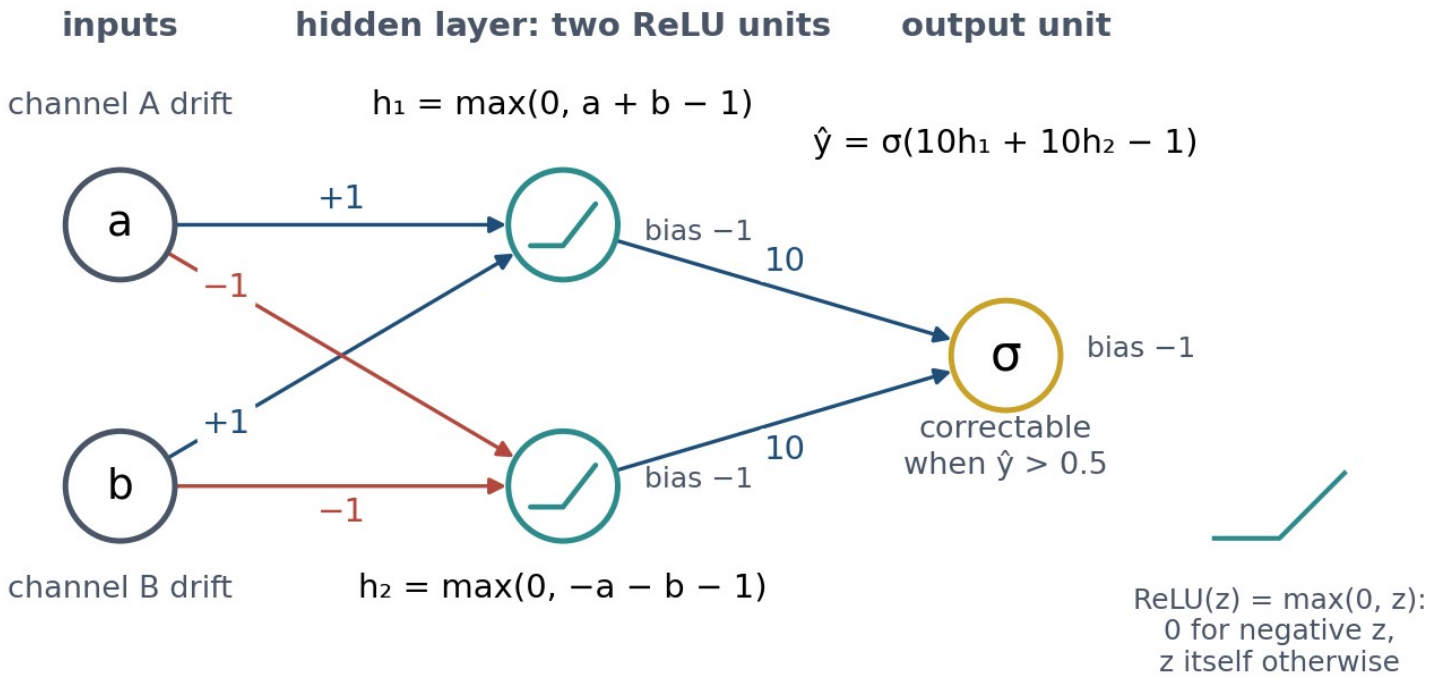
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The smallest problem that needs a hidden layer

- Correctable when the drifts a and b share a **sign**: the exclusive-or (XOR) of the two signs
- Logistic regression, all 800 sensors: exactly **0.5000**
- Correctable clouds have a + b near ±2; the others near 0
- The rule “|a + b| is large” needs **two boundaries**
- One line draws one boundary; two hidden units can draw two

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Two hidden units, drawn



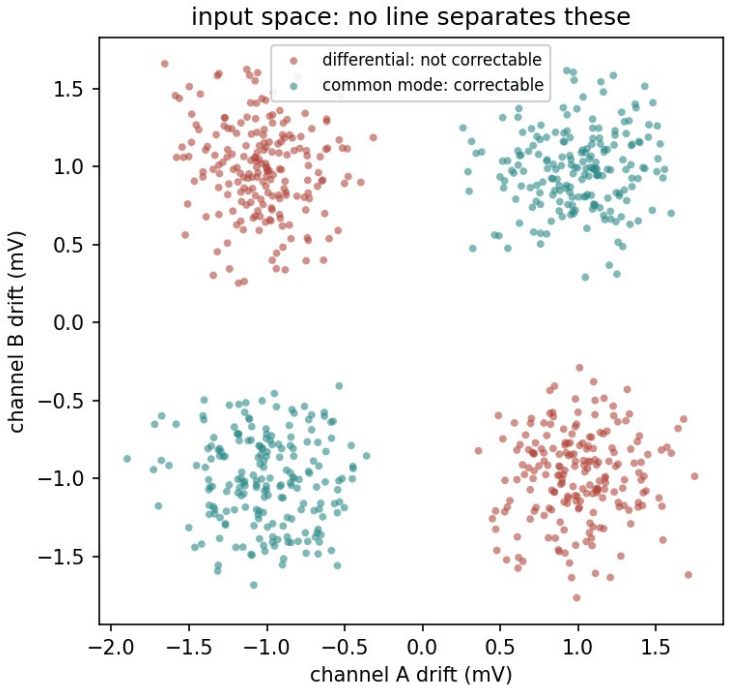
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

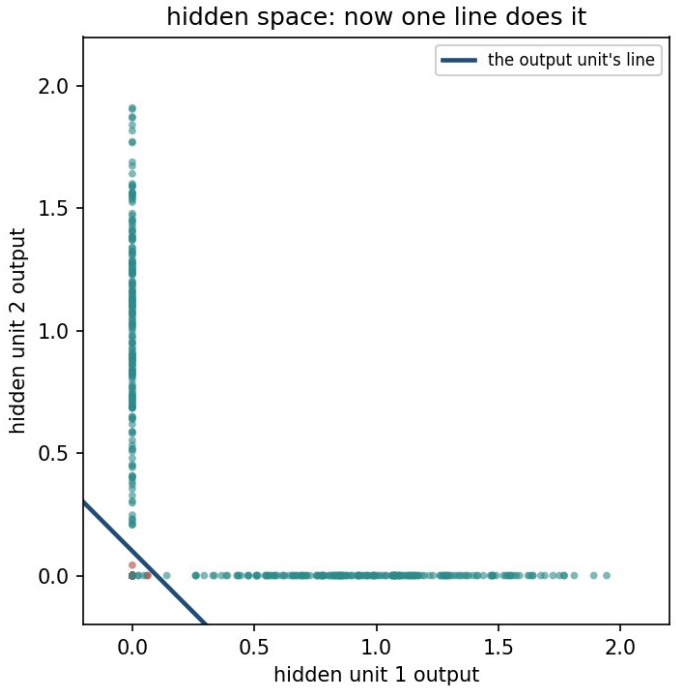
Two units, written down rather than trained

- Each hidden unit applies an **activation** to its sum z: the rectified linear unit (ReLU), max(0, z)
- $h_1 = \max(0, a + b - 1)$ and $h_2 = \max(0, -a - b - 1)$
- Output: $\hat{y} = \sigma(10h_1 + 10h_2 - 1)$
- $\hat{y}$ at centre (+1, +1): 0.9999; at (+1, −1): 0.2689
- All 800 sensors, **nothing trained**: accuracy **0.9938**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

What the hidden layer actually did





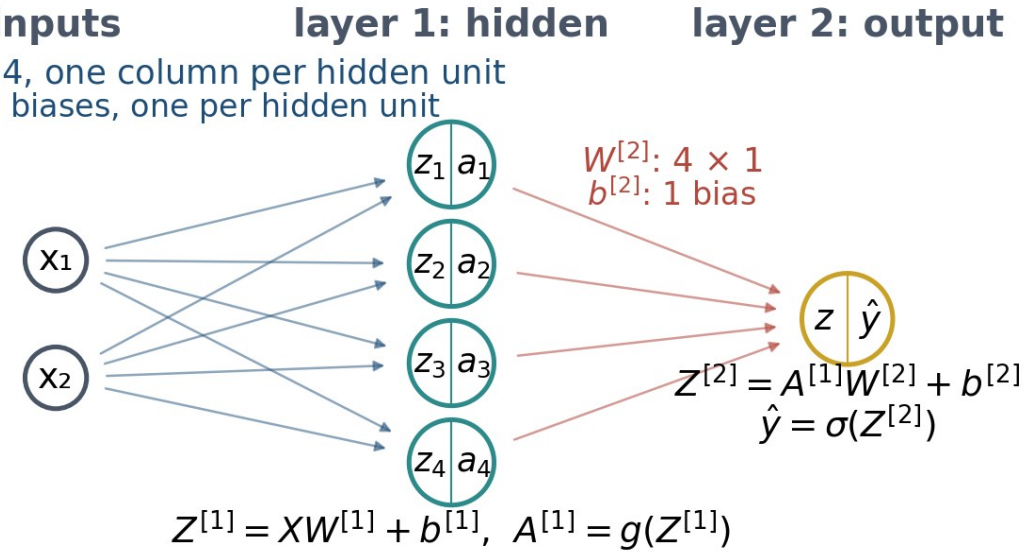
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The hidden layer classified nothing

- The hidden layer **moved the data**
- Correctable clouds went to (1, 0) and (0, 1); the rest to the origin
- There, the output unit’s **single line** is enough
- Learning such new coordinates from data is called **representation learning**
- Computing the output from the input, layer by layer: the **forward pass**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Forward propagation, drawn



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Forward propagation, written

$$A^{[1]} = g(XW^{[1]} + b^{[1]}), \quad \hat{y} = \sigma(A^{[1]}W^{[2]} + b^{[2]})$$

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

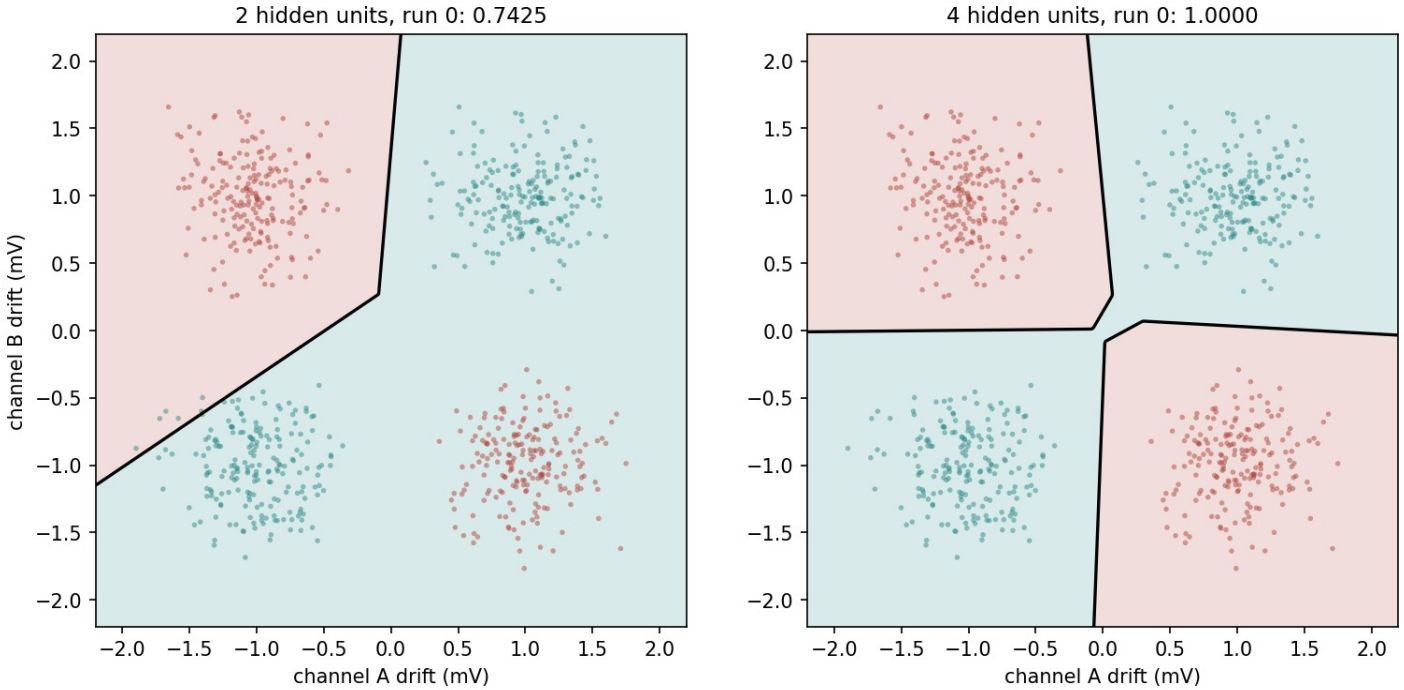
Letting training find the weights

- `MLPClassifier` (multilayer perceptron): one hidden layer, from random weights
- Drift, 800 sensors, 20 runs per size from different `random_state` values
- Runs reaching 0.99 accuracy: **6 of 20** with 2 units, 17 of 20 with 4, all 20 with 8
- In code:

```
MLPClassifier(hidden_layer_sizes=(4,))
```


UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Two units can; training often misses them

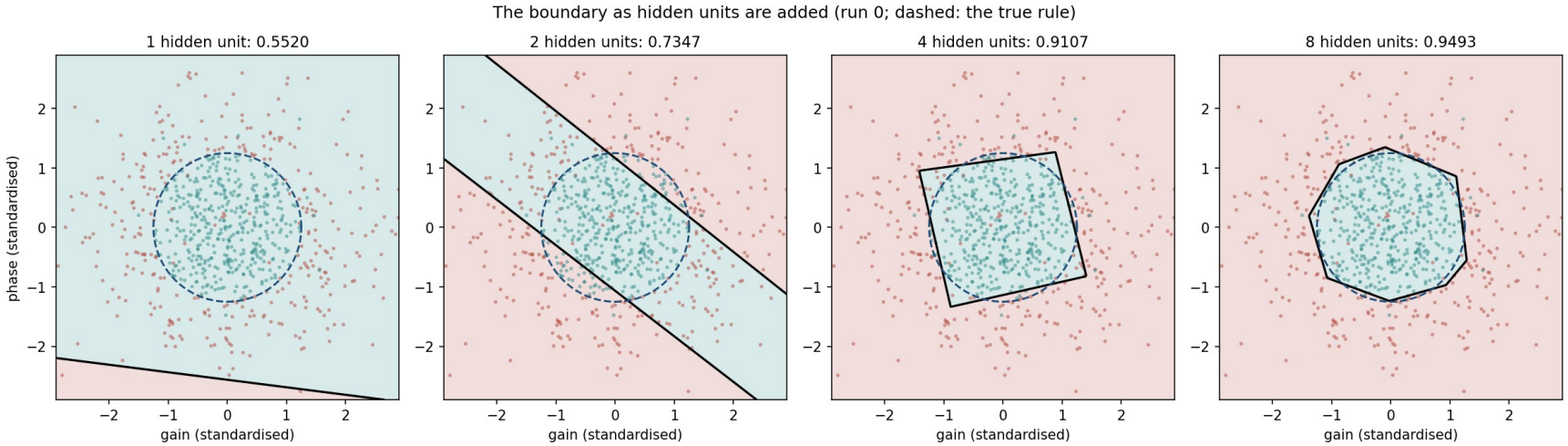


UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Acceptance test: how many hidden units?

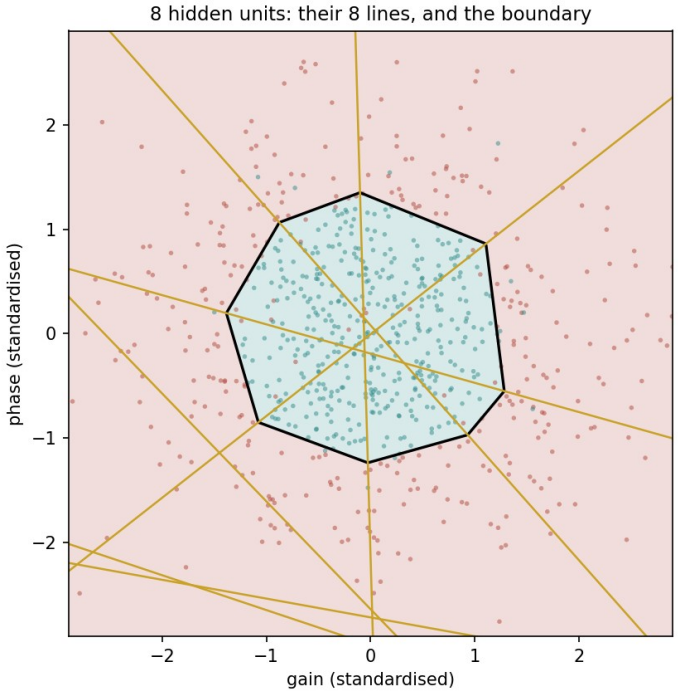
hidden units (750 test sensors)	mean of 5 runs	worst run	best run
1	0.6048	0.5480	0.6493
2	0.7392	0.7240	0.7693
3	0.8120	0.7227	0.9160
4	0.8821	0.7227	0.9507
8	**0.9445**	0.9387	0.9493
16	0.9549	0.9520	0.9600

The boundary at 1, 2, 4 and 8 units



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Each hidden unit is a line



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Notebook 1, live

section, and its slides	you call (→ the number to come back with)
look first, 10-11	`train_test_split`, `StandardScaler` → 2,250 to train, 750 to test
one line, 12-13	`LogisticRegression` → 0.5507 against 0.5480
one unit by hand, 7-9	NumPy only → ŷ = 0.8808 at (1, 1)
drift, hand-built network, 14-21	NumPy only, the two-unit network → 0.9938
training finds the weights, 22-23	`MLPClassifier`, 20 runs per size → 6 of 20 at two units
widths and hidden lines, 24-26	`MLPClassifier`, then `coefs_[0]` → 0.9445 at eight units
what it was worth, 24	one subtraction → 39.4 points

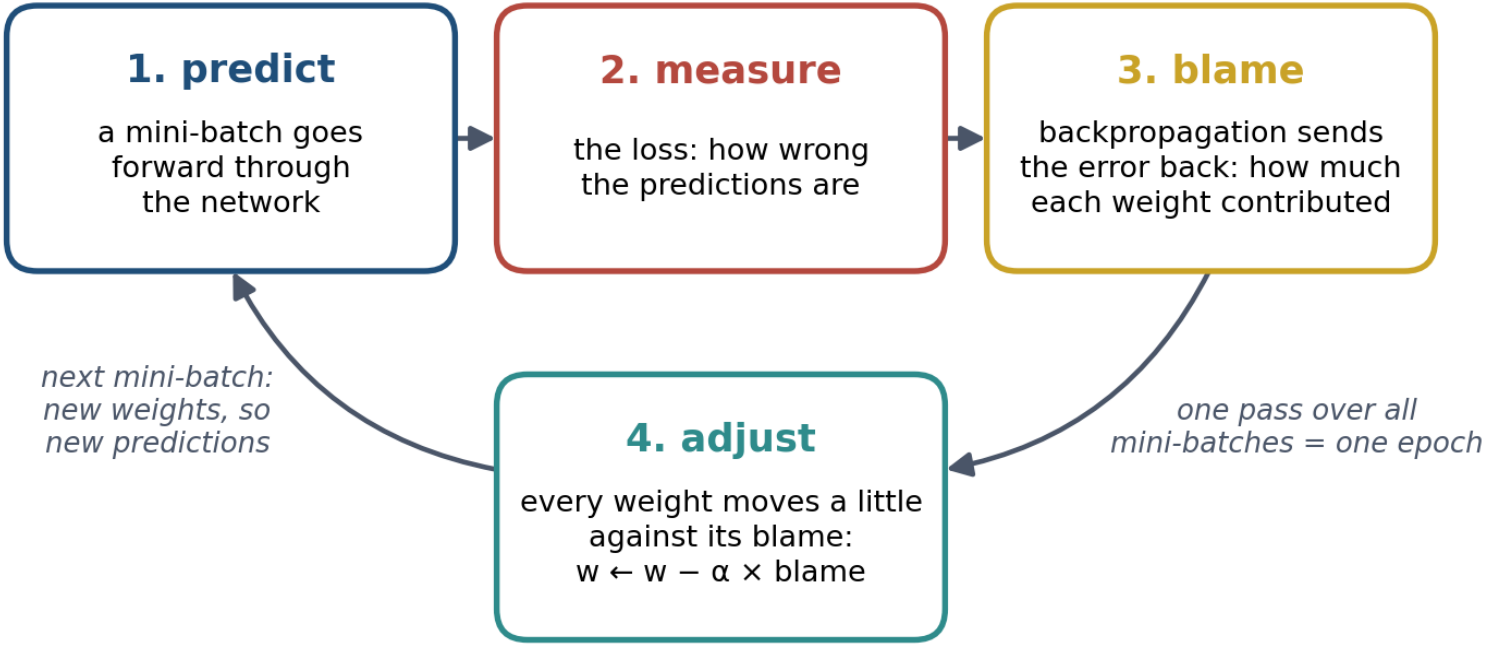
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Break

- Twelve minutes

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

How a network learns: the loop



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Mini-batches and epochs

- A **mini-batch**: a small random group of training examples, 32 in notebook 2
- Each round of the loop uses one mini-batch
- An **epoch**: one pass through all the mini-batches
- Digits: 1,077 training images in batches of 32 make **34 per epoch**
- 60 epochs make **2,040 rounds** of the loop

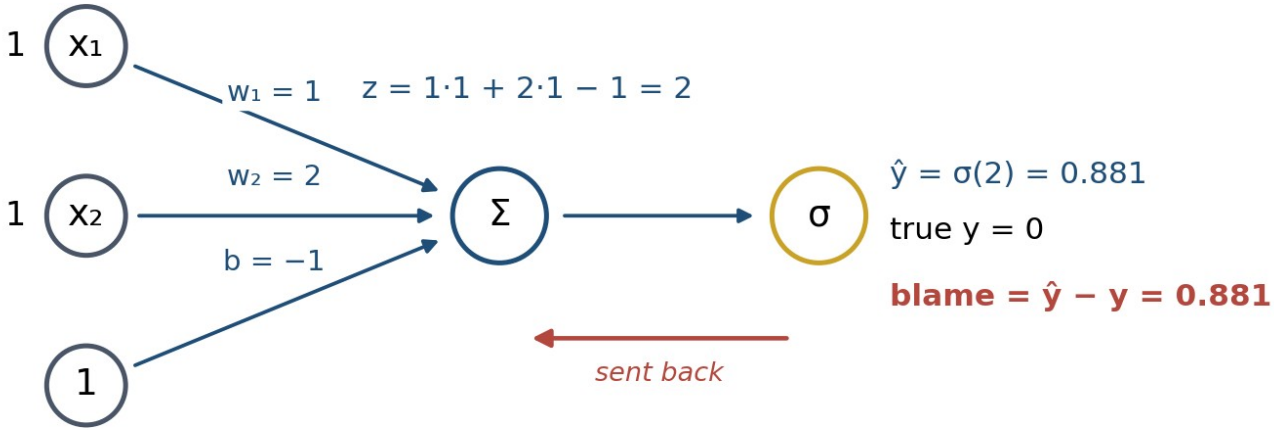
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The loss and the gradient step

- Each weight moves against its **gradient**: how fast the loss (lesson 4's cross-entropy) changes as that weight changes
- $w \leftarrow w - \alpha \cdot \partial L/\partial w$, with α the **learning rate** (lesson 3)
- A weight's gradient is its **blame**: its share of the error
- One sigmoid unit: blame $\hat{\mathbf{y}} - \mathbf{y}$ at the output; each weight's blame is $\hat{y} - y$ times its input

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

One step on one unit



each weight's blame = 0.881 × its input; step with α = 0.5:

$w_1 \to 0.560$, $w_2 \to 1.560$, $b \to -1.440$

new prediction: $\hat{y} = \sigma(0.679) = 0.663$, closer to the true 0

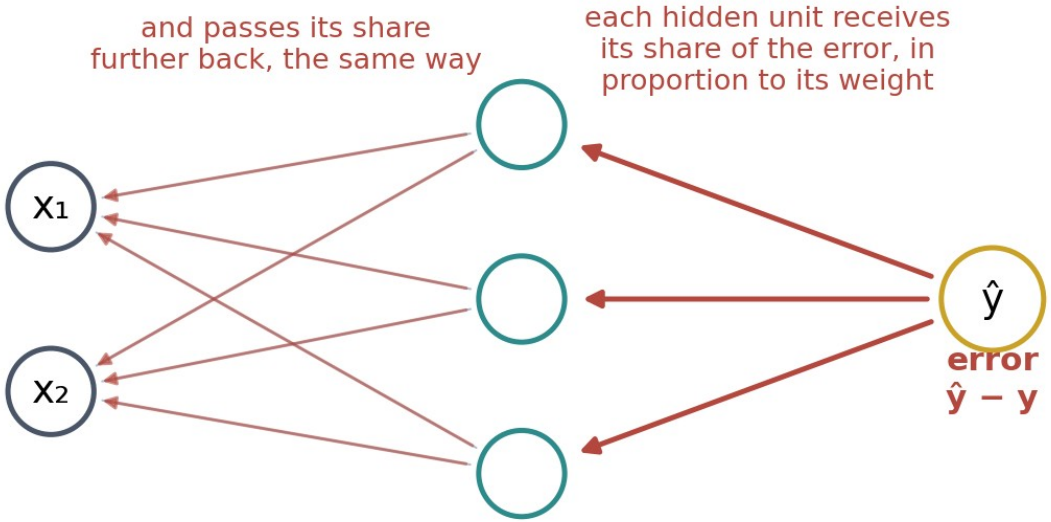
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Reading the step

- Unit $w = (1, 2)$, $b = -1$; input $x = (1, 1)$, true label $y = 0$
- Forward: $z = 2$, $\hat{y} = 0.881$, loss **2.127**
- Blame $\hat{y} - y = 0.881$ for every weight, since both inputs are 1
- Step with $\alpha = 0.5$: each weight down by 0.440, to 0.5596, 1.5596 and −1.4404
- New $\hat{y} =$ **0.663**, loss down to **1.089**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Backpropagation: blame flows back



one backward sweep gives every weight its blame, layer by layer, right to left

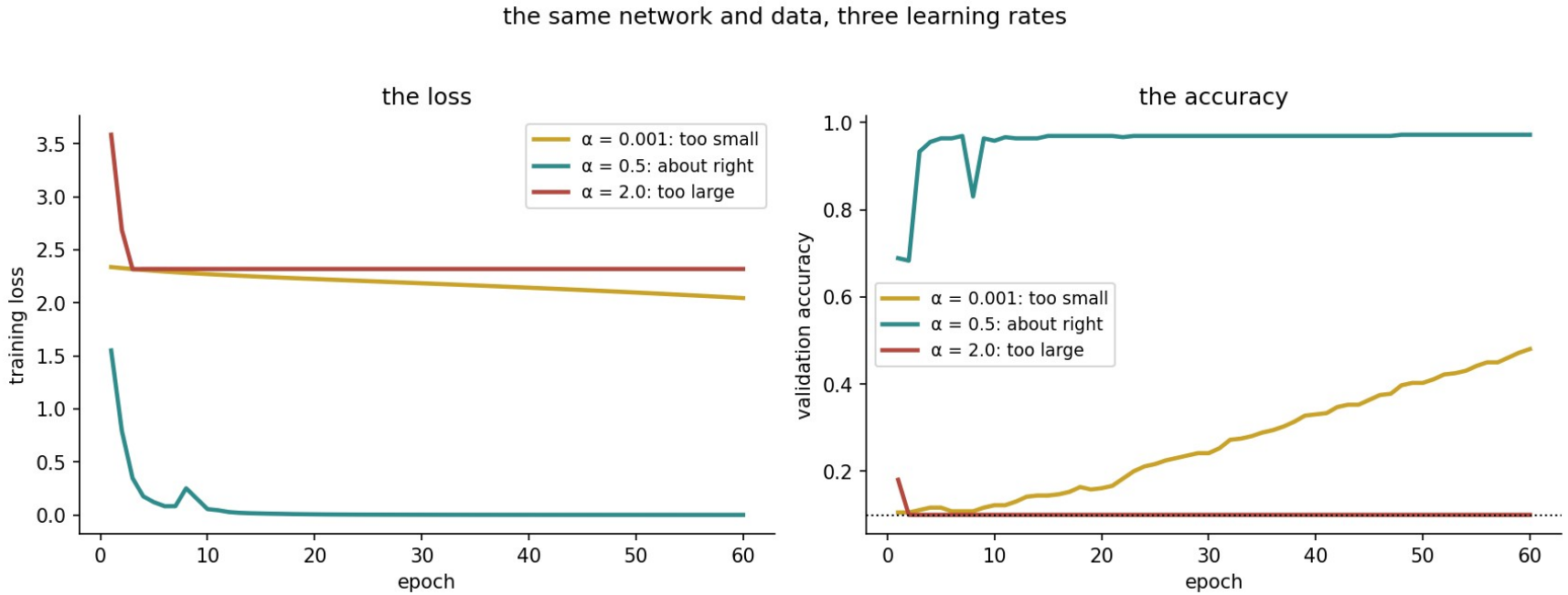
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

What backpropagation buys

- **Backpropagation**: the chain rule, applied from the output back
- Each hidden unit receives a share of the output’s error, in proportion to its weight
- It multiplies that share by its activation’s slope, then passes it back
- One backward sweep costs about one forward pass
- scikit-learn and Keras run it inside `fit`

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The learning rate, three regimes



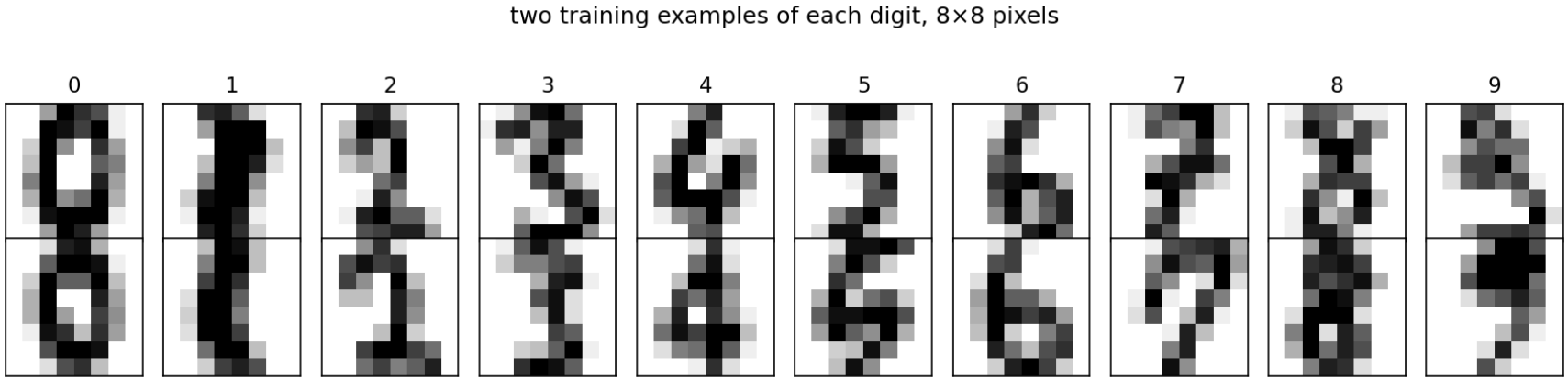
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Predictable mistake: a rate too small

- Two layers of 64, stochastic gradient descent (SGD: one step per mini-batch), 60 epochs, one run per rate
- Validation digits: 0.4806 at $\alpha = 0.001$, **0.9722** at 0.5, 0.1000 at 2.0
- A rate too small looks like **a network too small**
- **Try rates a factor of ten apart** first
- In code: `keras.optimizers.SGD(0.5)`

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Ten digits, eight by eight pixels



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

A network in Keras: 2,410 parameters

- **Keras**: the network library of lessons 9 and 10
- `keras.Sequential`: the layers in order, `keras.Input` then two `layers.Dense`
- A **dense** layer: every unit sees every input (32 ReLU, then 10 softmax)
- `compile` picks `Adam` (a refined gradient descent); `fit` trains
- Parameters: 64 × 32 + 32 = 2,080, plus 32 × 10 + 10 = 330

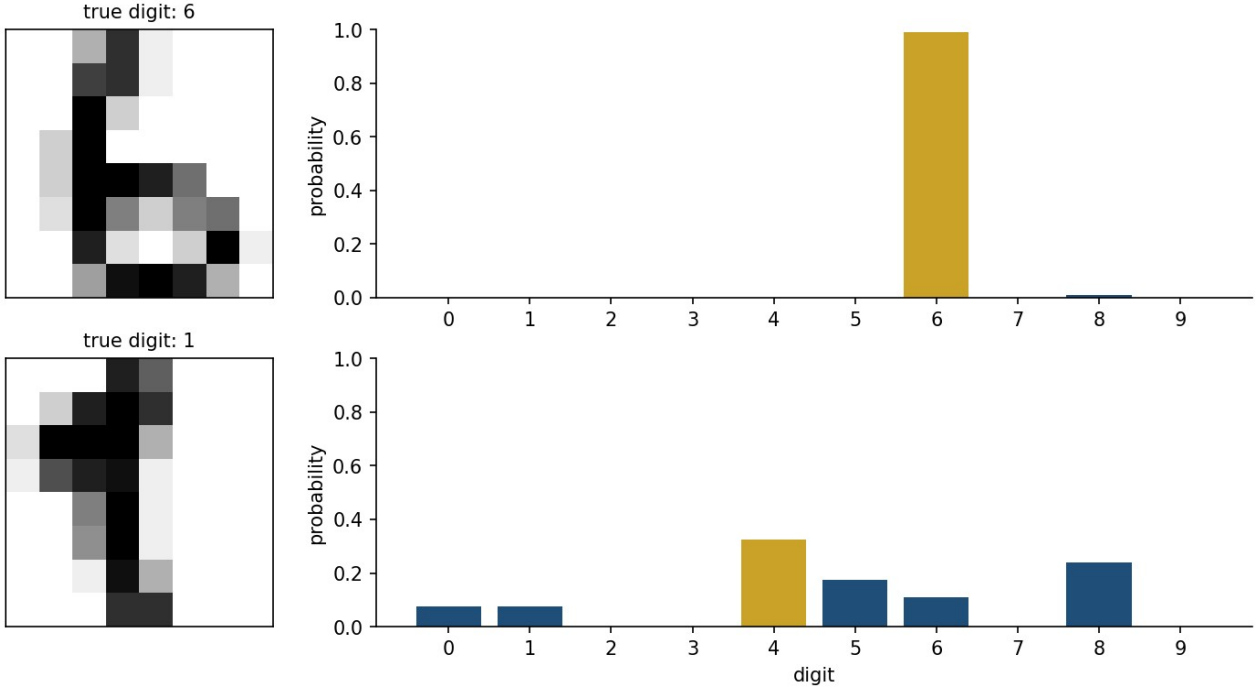
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Softmax

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{10} e^{z_j}}$$

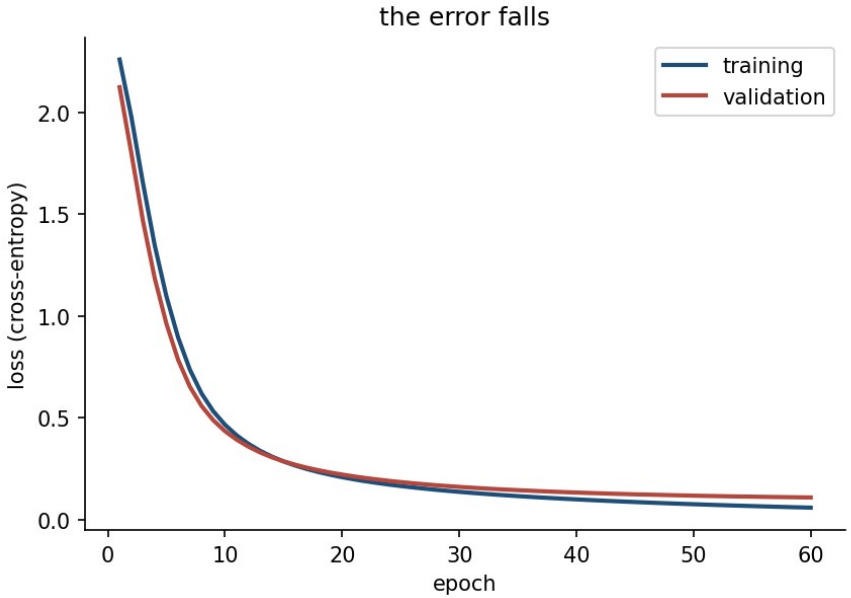
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

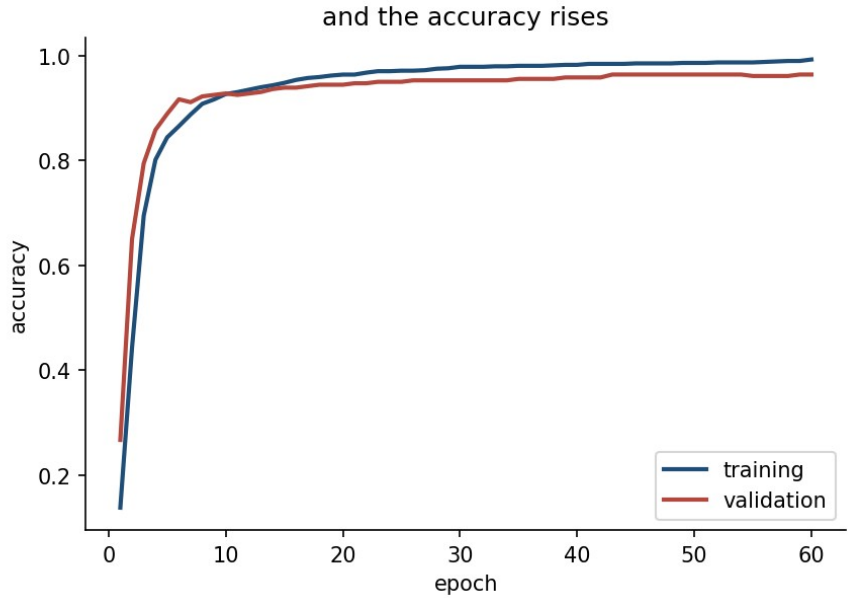
Ten scores become ten probabilities



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

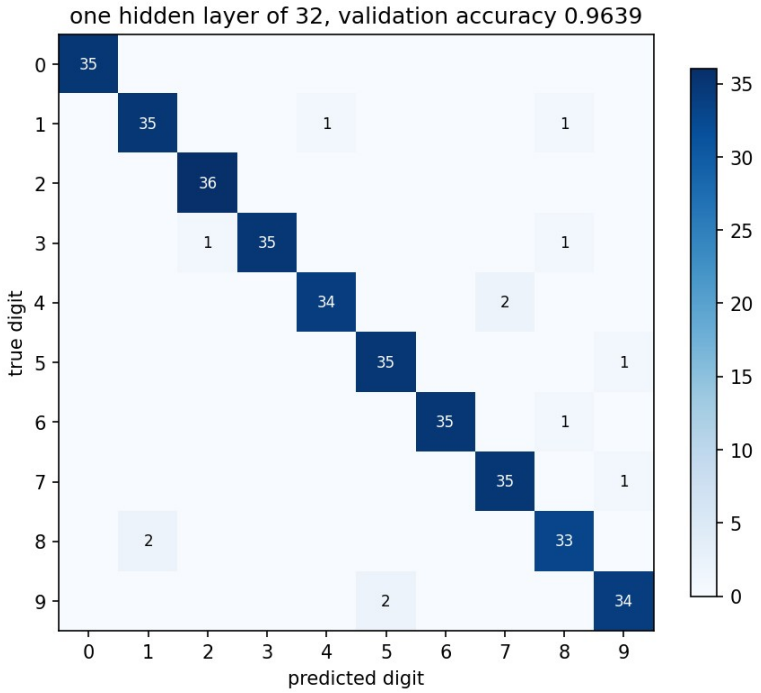
Training, watched





UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Where it goes wrong



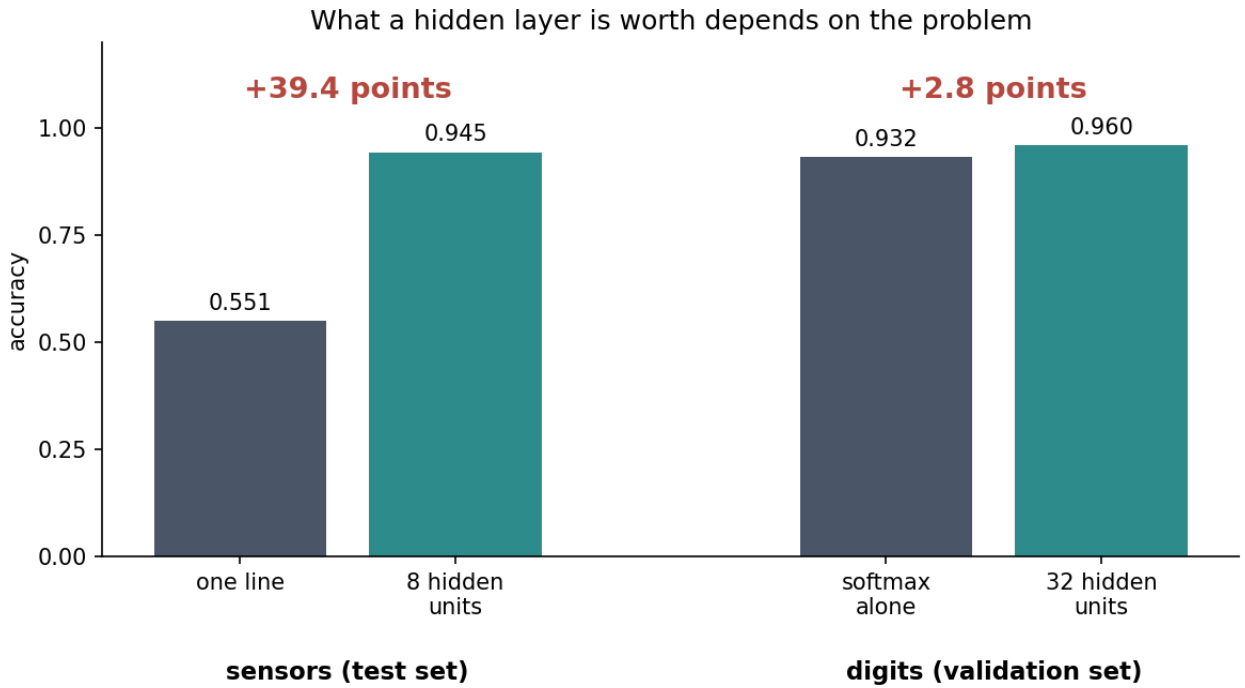
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Is the hidden layer worth it here?

architecture, on the 360 validation digits	parameters	accuracy, mean of 3 runs	standard deviation, 3 runs
softmax alone, no hidden layer	650	0.9324	0.0035
one hidden layer of 32	2,410	**0.9602**	0.0052
one hidden layer of 64	4,810	0.9611	0.0045

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

39 points against 2.8



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

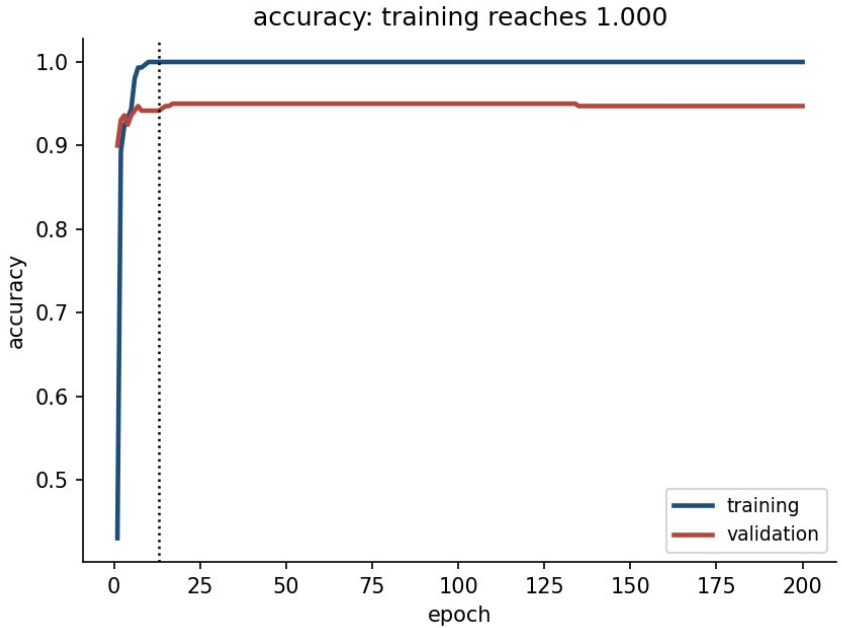
When to reach for a hidden layer

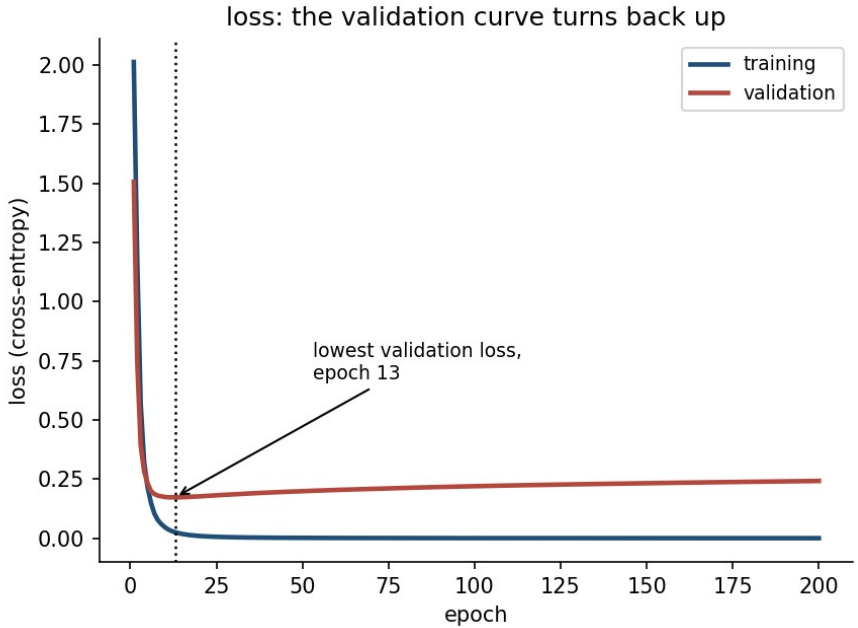
- Sensors: the two offsets act **together**
- Digits: each pixel pushes **on its own**
- The predictable mistake: adding a hidden layer every time, **by reflex**
- Digits: 2,410 parameters against 650, for 2.8 points
- **Fit the linear model first**: its gap to a network says whether a network is needed

Notebook 2, live

section, and its slides	you call (→ the number to come back with)
the digits, 38	`train_test_split`, twice → 1,077, 360 and 360 images
building and training, 29-30, 39, 42	`keras.Sequential`, `keras.Input`, `layers.Dense`, `Adam`, `fit` → 2,410 parameters, 0.9639
softmax and confusion matrix, 40-43	`model.predict`, `confusion_matrix` → 13 of 360 wrong
is the hidden layer worth it, 44	three architectures, 3 runs each → 0.9324, 0.9602, 0.9611
the number, 45-46	`LogisticRegression`, `MLPClassifier` on the sensors again → 39.4 and 2.8 points; 4.7 on another split

A network that memorises





UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Reading the two curves

- One run: two layers of 512 on **300 digits**: 301,066 parameters
- Training accuracy **1.000**: the 300 answers are stored
- Validation loss lowest at **epoch 13**, 1.4 times that by epoch 200
- Validation accuracy stays near 0.95 (0.9472 at epoch 200), while the network grows more confident about the answers it gets wrong

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Two remedies

- **Early stopping**: stop after 25 epochs with no better validation loss; keep the best weights
- **Dropout**: switch off a random 40% of hidden units on each training mini-batch
- In code: `EarlyStopping(patience=25, restore_best_weights=True)`
- In code: `layers.Dropout(0.4)`

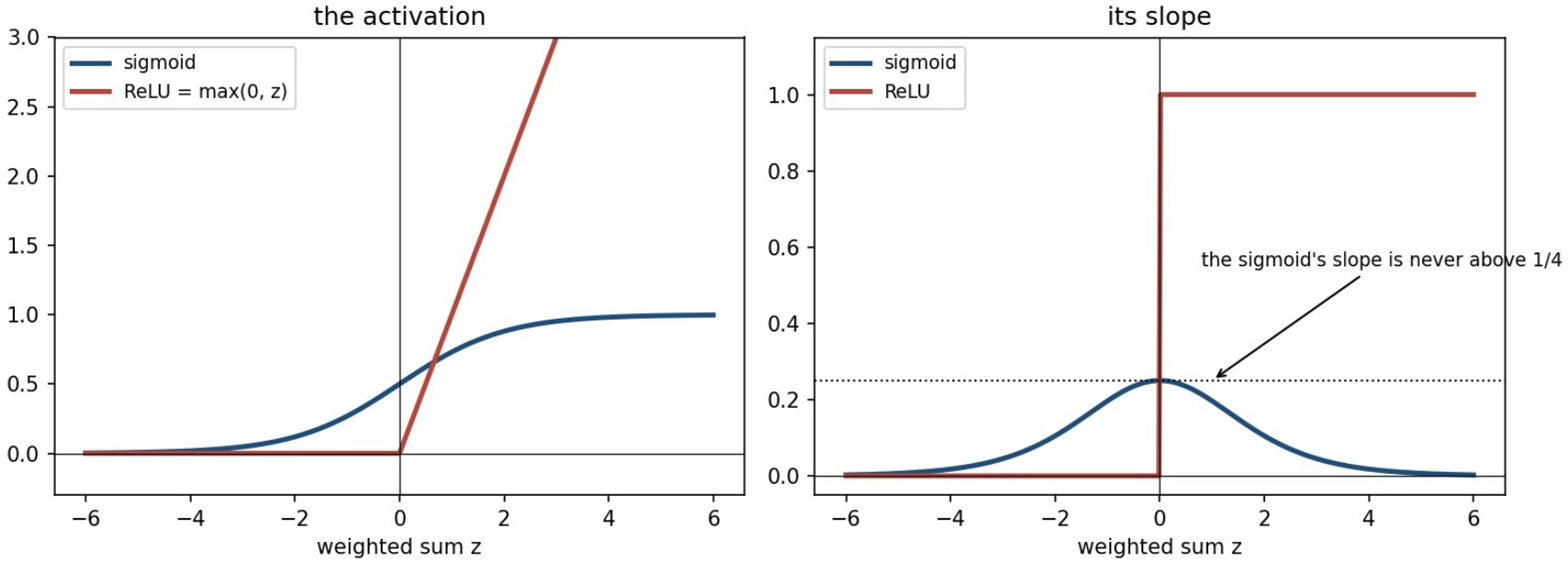
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Two remedies, measured

method, two layers of 512 on 300 digits	test accuracy, mean of 3 runs	standard deviation, 3 runs	epochs run
200 epochs, no remedy	0.9426	0.0013	200
early stopping	0.9463	0.0035	39
early stopping + dropout 0.4	0.9481	0.0035	50.3

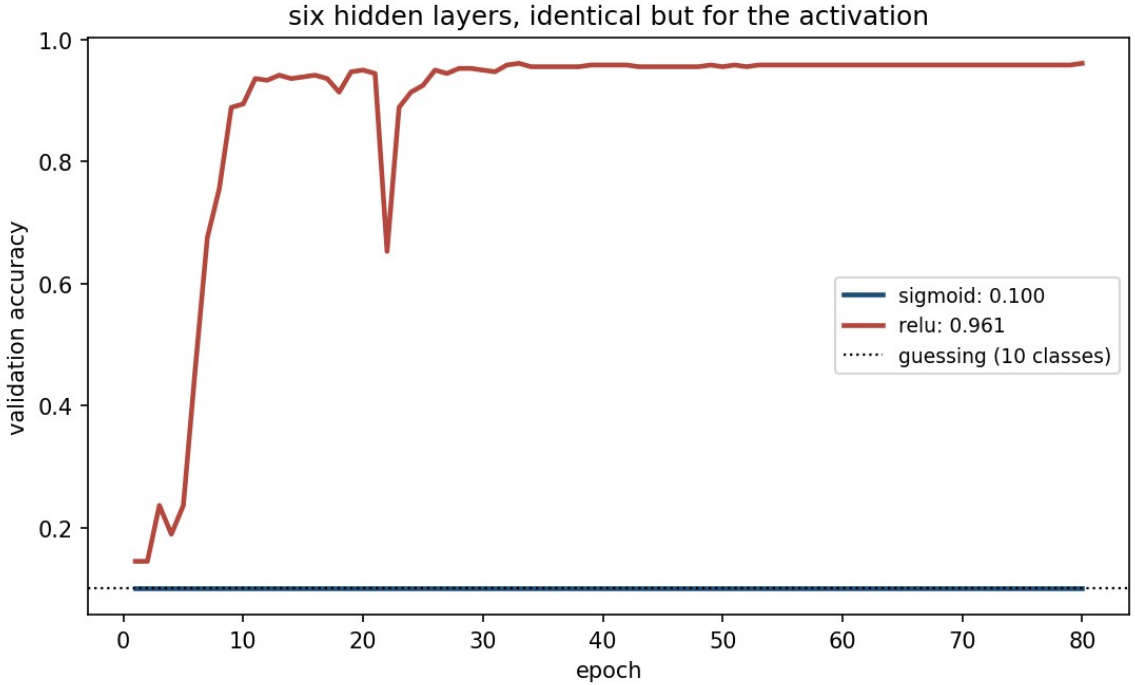
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Activations and their slopes



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Six sigmoid layers never learn



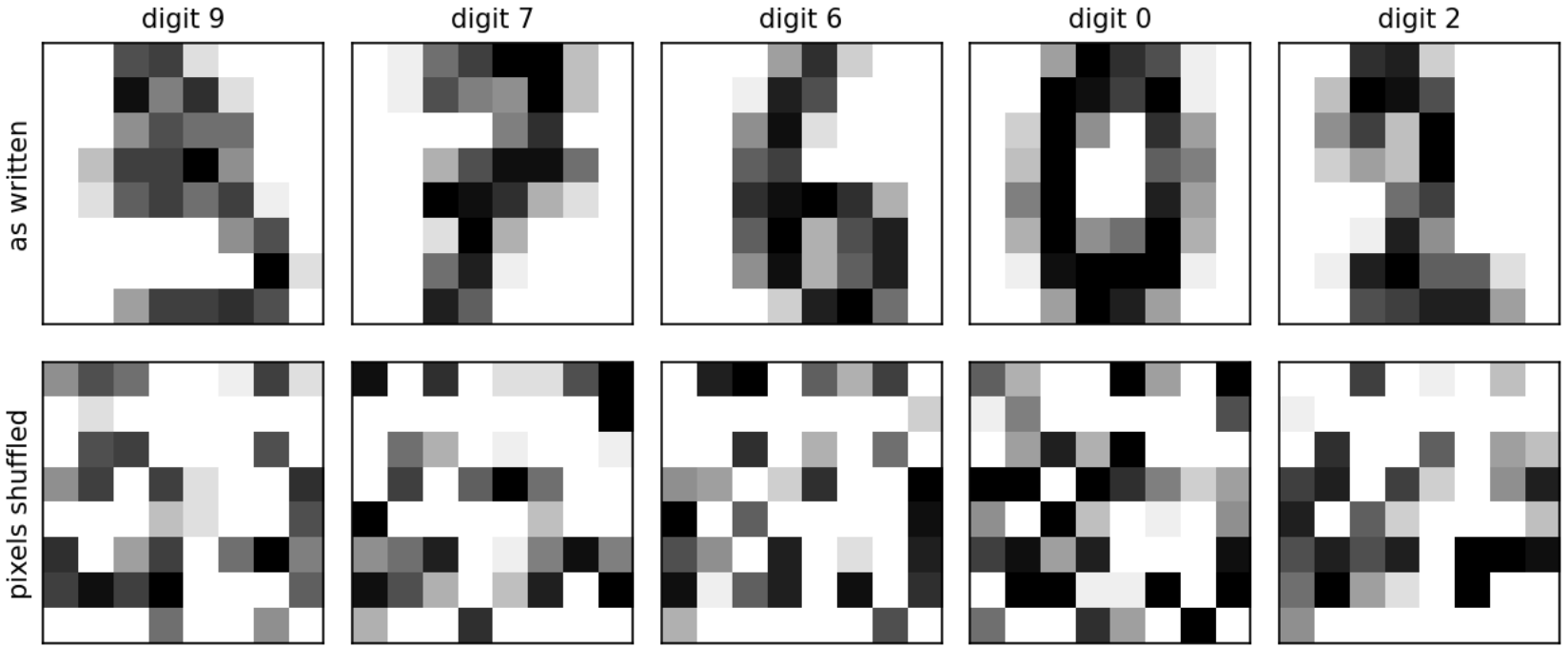
UNIVERSITÀ DEGLI STUDI DELL'AQUILA

The vanishing gradient

- Each layer multiplies the blame by its **slope**
- Slope at most 1/4 for the sigmoid; 1 for an active ReLU
- Six sigmoid layers: at most $0.25^6 = 1/4096$ of the blame reaches the first
- Six layers of 32, one run: sigmoid **0.1000**, ReLU **0.9611**
- This is the **vanishing gradient**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Shuffle the pixels



UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Nothing notices

- One fixed reordering of the 64 pixels
- Hidden layer of 32, 3 runs: **0.9602** as written, **0.9630** shuffled
- A difference well inside the run-to-run spread
- A dense layer never uses which pixel is next to which
- **Permute the pixels consistently and nothing here notices.**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Notebook 3, live

section, and its slides	you call (→ the number to come back with)
the learning rate, 36-37	`keras.optimizers.SGD` at three rates → 0.9722 at α = 0.5
memorising, 48-49	two `Dense` layers of 512 on 300 digits → validation loss lowest at epoch 13
two remedies, 50-51	`EarlyStopping`, `Dropout(0.4)`, 3 runs each → 0.9426, 0.9463, 0.9481
sigmoid against ReLU, 52-54	six `Dense` layers of 32 → 0.1000 against 0.9611
shuffled pixels, 55-56	one fixed permutation of the 64 pixels → 0.9602 against 0.9630

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

What to take away

- **One neuron is one line**; a **hidden layer moves the data**
- **Learning is a loop**: predict, measure the loss, share out the blame, adjust the weights
- Low accuracy? First try learning rates ten times apart
- **Fit the linear model first**: its gap to a network says if one is needed

39 points against 2.8

- One hidden layer on the acceptance test: **+39.4 points** (8 units, mean of 5 runs, 750 test sensors)
- One hidden layer on the digits: **+2.8 points** (32 units, mean of 3 runs, 360 validation images; 4.7 on another split)
- On the sensors the offsets act **together**; on the digits each pixel acts mostly **on its own**
- What a network is worth depends on the **shape of the problem**

UNIVERSITÀ DEGLI STUDI DELL'AQUILA

Homework

- **Exercise 9**, discussed at the start of lesson 10, **Friday 18 December**
- `Exercises/09_neural_networks.md`
- Two batches from Meridian’s two final-test stations: on which one is a hidden layer worth it, and why?
- Next week starts from today’s last finding: permute the pixels consistently and nothing here notices

UNIVERSITÀ DEGLI STUDI DELL'AQUILA